K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

School of Engineering and Technology

Capstone Project

Smart City Traffic Management and Route Optimization System

Submitted By:

Tamanna

2401201138

BCA (AI & DS) - A

Submitted To:

Ms. Aarti Sangwan

1. Introduction

Modern cities depend on road networks that must move people quickly and safely. A road network can be represented as a weighted graph in which every intersection is a vertex and every road is an edge whose weight is the distance or travel time. Once the city is a graph, classic algorithms can plan routes, react to congestion and design cheaper infrastructure.

Objective. This capstone project builds a Smart City Traffic Management and Route Optimization System in Python with a Streamlit interface. It implements and compares Dijkstra, Bellman-Ford, BFS, DFS, Prim and Kruskal on a sample city of 8 intersections and 12 roads, and benchmarks them on networks of 10, 25, 50 and 100 intersections.

Module	File	Algorithm
Road network model	graph_model.py	Adjacency list and matrix
Route planning	route_planner.py	Dijkstra
Traffic analysis	traffic_analysis.py	Bellman-Ford
Connectivity	connectivity_module.py	BFS, DFS
Infrastructure planning	mst_planner.py	Prim, Kruskal
Comparison	benchmark.py	Time and memory tests

2. Road Network Modelling

The RoadNetwork class stores intersections as nodes and roads as undirected weighted edges. It validates every road (two different intersections and a positive cost) and can produce both an adjacency list and an adjacency matrix.

Code (graph_model.py)

```
class RoadNetwork:
    def __init__(self):
        self.nodes = []
        self.edges = {}  # (u, v) with u < v  ->  weight


    def add_road(self, u, v, w):
        if u == v:
            raise ValueError("A road must connect two different intersections.")
        if w <= 0:
            raise ValueError("Travel cost must be positive.")
        self.add_intersection(u)
        self.add_intersection(v)
        self.edges[tuple(sorted((u, v)))] = float(w)

    def adjacency_list(self):
        adj = {n: [] for n in self.nodes}
        for (u, v), w in self.edges.items():
            adj[u].append((v, w))
            adj[v].append((u, w))
        for n in adj:
            adj[n].sort()
        return adj

    def adjacency_matrix(self):
        df = pd.DataFrame(0.0, index=self.nodes, columns=self.nodes)
```

```
        for (u, v), w in self.edges.items():
            df.loc[u, v] = w
            df.loc[v, u] = w
        return df
```

Output

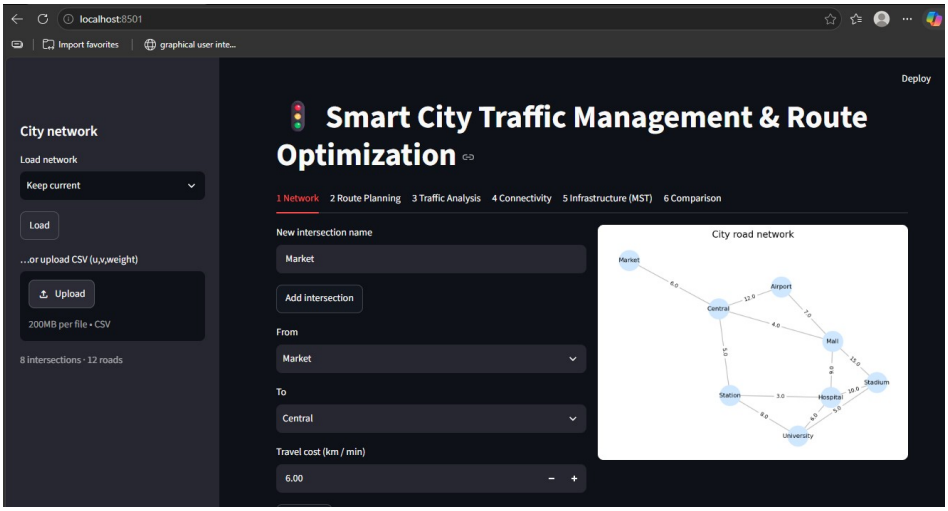


Figure 1: Network tab: add intersections and roads, city road network graph

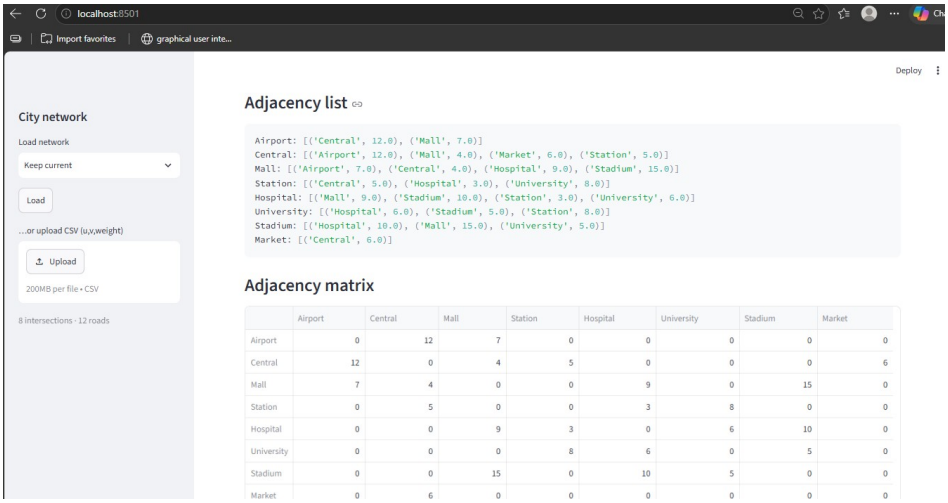


Figure 2: Adjacency list and adjacency matrix of the network

The sample city has 8 intersections (Airport, Central, Mall, Market, Station, Hospital, University and Stadium) joined by 12 roads.

3. Route Planning using Dijkstra

Dijkstra's algorithm uses a priority queue to always expand the nearest unvisited intersection. Every distance update is stored in a step log so the working can be shown in the interface.

Code (route_planner.py)

```
def dijkstra(g, source, target=None):
    """Return (dist, prev, steps). steps logs every distance update."""
    adj = g.adjacency_list()
    dist = {n: float("inf") for n in g.nodes}
    prev = {n: None for n in g.nodes}
    dist[source] = 0
    pq = [(0, source)]
    visited = set()
    steps = []
    while pq:
        d, u = heapq.heappop(pq)
        if u in visited:
            continue
        visited.add(u)
        steps.append({"Visit": u, "Dist(u)": d, "Updated": "-", "New Dist": "-", "Via":
"-"})
        if u == target:
            break
        for v, w in adj[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                prev[v] = u
                heapq.heappush(pq, (dist[v], v))
                steps.append({"Visit": u, "Dist(u)": d, "Updated": v, "New Dist":
dist[v], "Via": u})
    return dist, prev, steps
```

Output

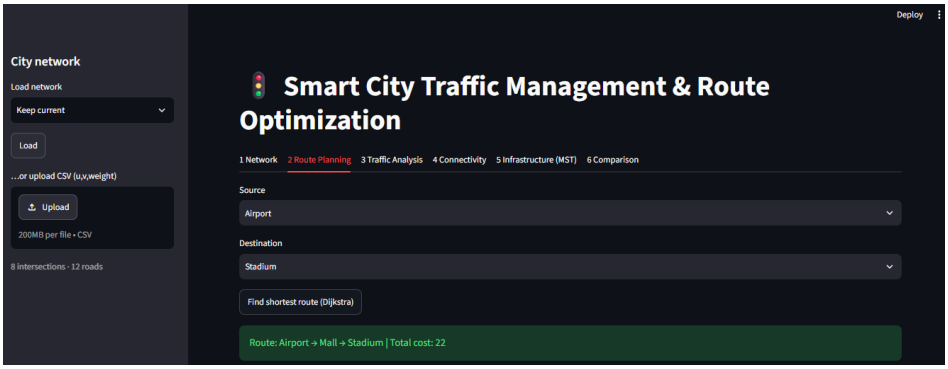


Figure 3: Route planning: shortest route found by Dijkstra

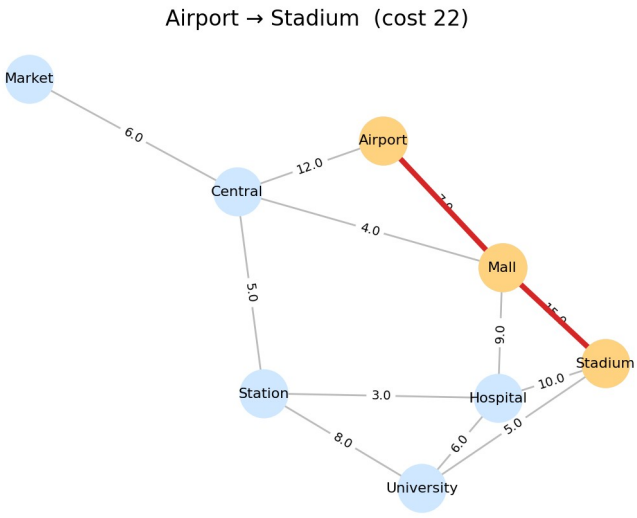


Figure 4: Shortest path highlighted on the city graph

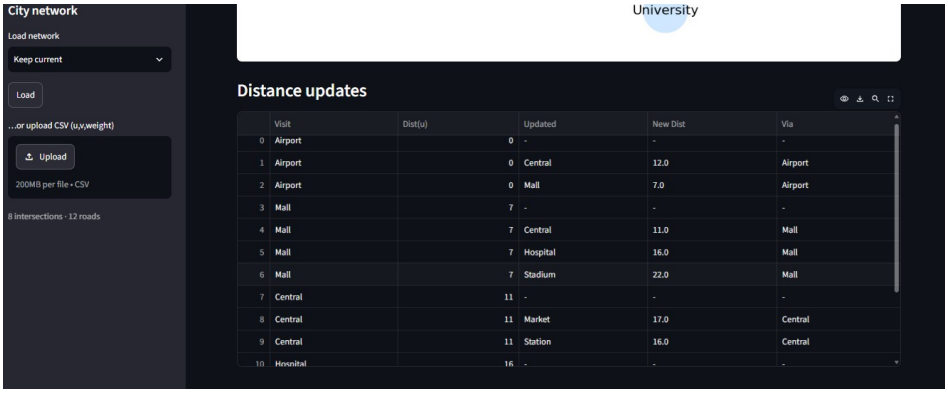


Figure 5: Dijkstra distance-update table

Result: the shortest route from Airport to Stadium is Airport → Mall → Stadium with a total cost of 22.

4. Traffic Analysis using Bellman-Ford

Bellman-Ford relaxes every road repeatedly, so it also works when weights change. To simulate traffic, apply_congestion() multiplies the weight of selected roads and compare_routes() checks whether the best route changes.

Code (traffic_analysis.py)

```
def bellman_ford(g, source):
    """Each undirected road = two directed arcs. Returns (dist, prev, log)."""
    arcs = []
    for (u, v), w in g.edges.items():
        arcs += [(u, v, w), (v, u, w)]
    dist = {n: float("inf") for n in g.nodes}
    prev = {n: None for n in g.nodes}
    dist[source] = 0
    log = []
    for i in range(1, len(g.nodes)):
        changed = False
        for u, v, w in arcs:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                prev[v] = u
                changed = True
                log.append({"Pass": i, "Relaxed": f"{u} -> {v}", "New Dist": dist[v]})
        if not changed:
            break
    return dist, prev, log

def apply_congestion(g, changes):
    """changes: list of (u, v, multiplier). Returns a new congested network."""
    g2 = g.copy()
    for u, v, factor in changes:
        key = tuple(sorted((u, v)))
        g2.edges[key] = g.edges[key] * factor
    return g2
```

Output

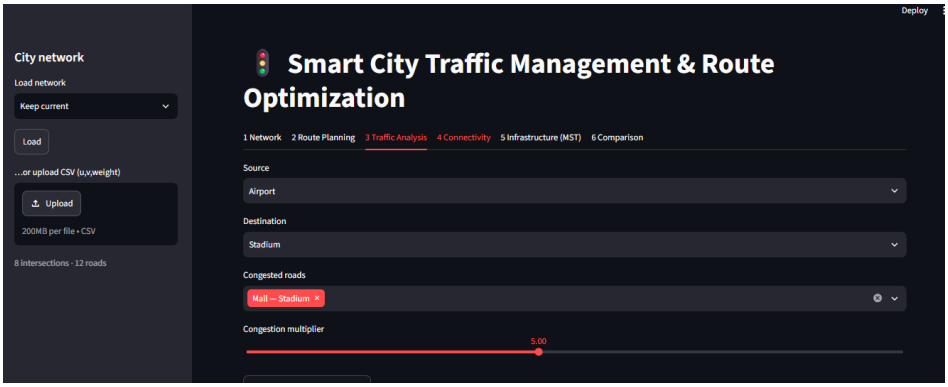


Figure 6: Traffic analysis: selecting the congested road and multiplier

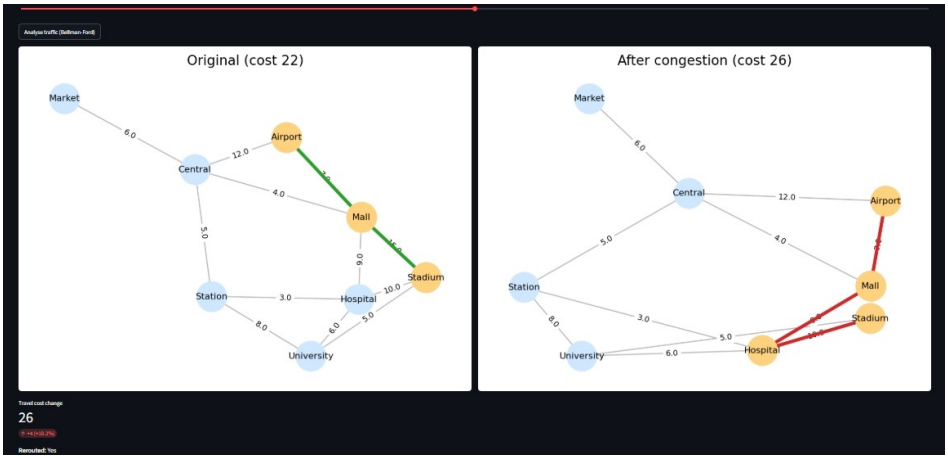


Figure 7: Original network (cost 22) versus congested network (cost 26)

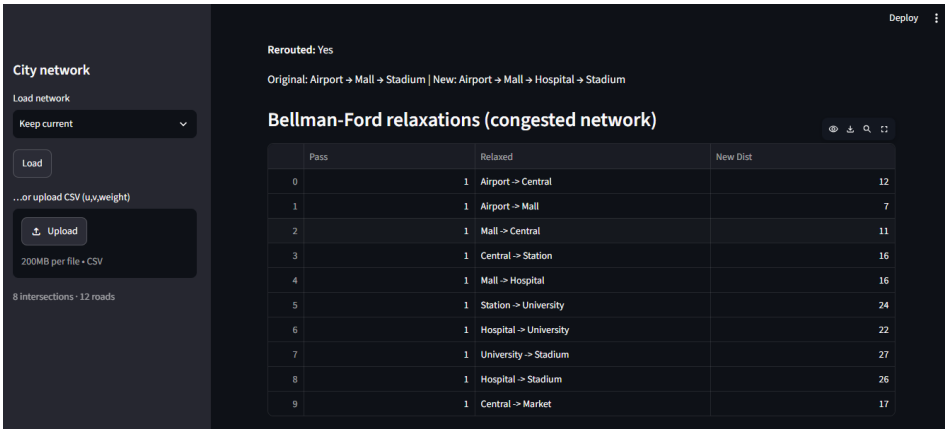


Figure 8: Bellman-Ford relaxation table for the congested network

Result: after multiplying the Mall–Stadium road by 5, the system reroutes to Airport → Mall → Hospital → Stadium. The cost rises from 22 to 26, an increase of about 18%.

5. Connectivity using BFS and DFS

BFS explores the network level by level using a queue, while DFS goes as deep as possible using recursion. Both are used to find which intersections can be reached from a starting point.

Code (connectivity_module.py)

```
def bfs(g, start):
    adj = g.adjacency_list()
    seen, order, q = {start}, [], deque([start])
    while q:
        u = q.popleft()
        order.append(u)
        for v, _ in adj[u]:
            if v not in seen:
                seen.add(v)
                q.append(v)
    return order

def dfs(g, start):
    adj = g.adjacency_list()
    seen, order = set(), []

    def visit(u):
        seen.add(u)
        order.append(u)
        for v, _ in adj[u]:
            if v not in seen:
                visit(v)
    visit(start)
    return order
```

Output

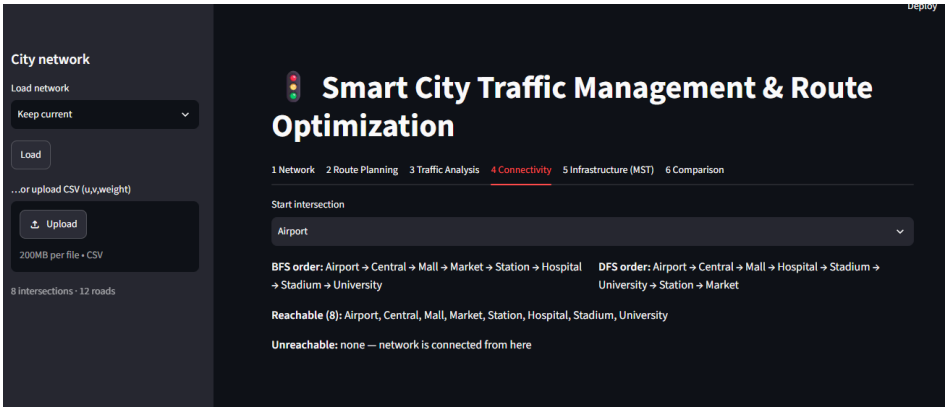


Figure 9: Connectivity tab: BFS and DFS traversal from Airport

- **BFS order:** Airport, Central, Mall, Market, Station, Hospital, Stadium, University
- **DFS order:** Airport, Central, Mall, Hospital, Stadium, University, Station, Market

All 8 intersections are reachable, so the network is fully connected.

6. Infrastructure Planning using MST

A minimum spanning tree connects every intersection with the lowest total road cost. Prim grows the tree from a starting node using a priority queue; Kruskal sorts all roads and adds the cheapest one that does not form a cycle (union-find).

Code (mst_planner.py)

```
def prim(g, start=None):
    adj = g.adjacency_list()
    start = start or g.nodes[0]
    seen, mst, total = {start}, [], 0.0
    pq = [(w, start, v) for v, w in adj[start]]
    heapq.heapify(pq)
    while pq and len(seen) < len(g.nodes):
        w, u, v = heapq.heappop(pq)
        if v in seen:
            continue
        seen.add(v)
        mst.append((u, v, w))
        total += w
        for x, wx in adj[v]:
            if x not in seen:
                heapq.heappush(pq, (wx, v, x))
    return mst, total

def kruskal(g):
    parent = {n: n for n in g.nodes}

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    mst, total = [], 0.0
    for (u, v), w in sorted(g.edges.items(), key=lambda e: e[1]):
        ru, rv = find(u), find(v)
        if ru != rv:
            parent[ru] = rv
            mst.append((u, v, w))
            total += w
    return mst, total
```

Output

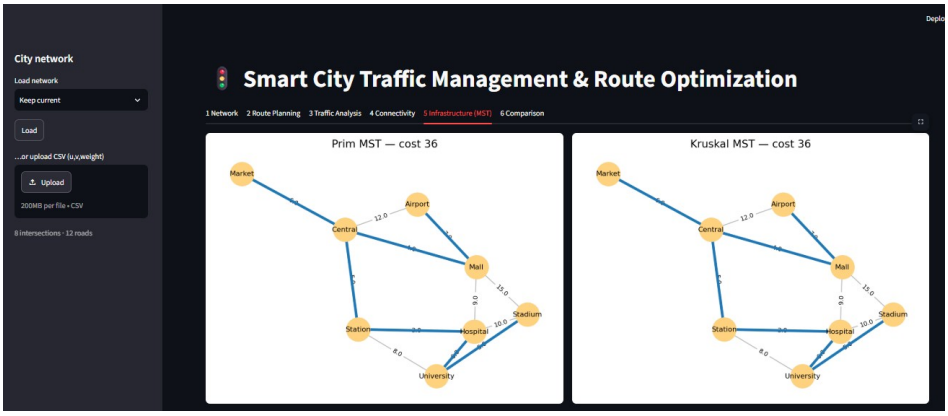


Figure 10: Prim MST and Kruskal MST (cost 36 each)

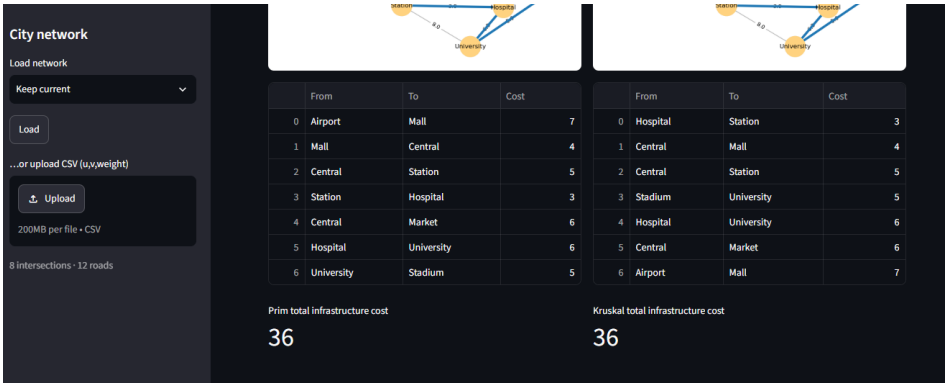


Figure 11: Roads selected by Prim and Kruskal

Result: both algorithms give the same minimum infrastructure cost of 36.

7. Performance Comparison

benchmark.py runs every algorithm on random networks of 10, 25, 50 and 100 intersections. It measures the average time over 20 runs and the peak memory using tracemalloc.

Code (benchmark.py)

```
def timed(fn, *a, repeat=20):
    tracemalloc.start()
    t0 = time.perf_counter()
    for _ in range(repeat):
        out = fn(*a)
    ms = (time.perf_counter() - t0) / repeat * 1000
    mem = tracemalloc.get_traced_memory()[1] / 1024
    tracemalloc.stop()
    return out, ms, mem
```

Output

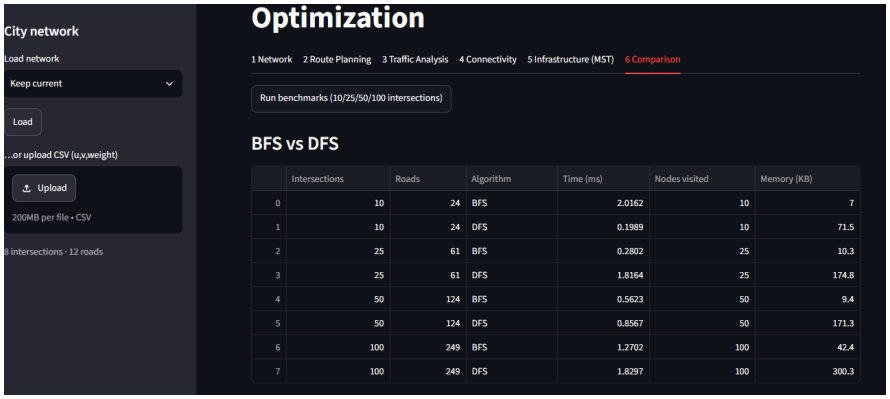


Figure 12: BFS versus DFS benchmark

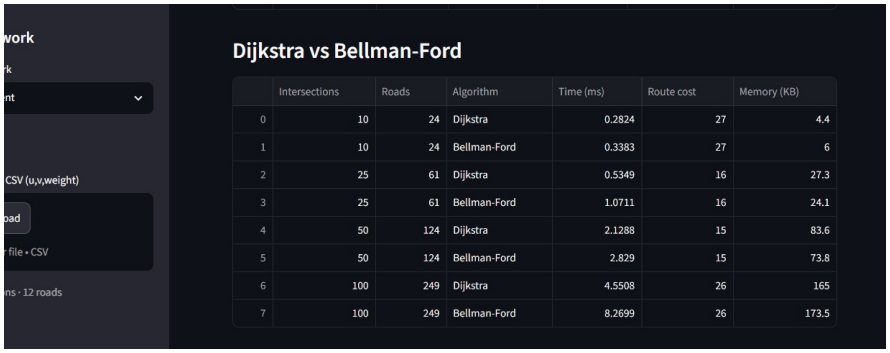


Figure 13: Dijkstra versus Bellman-Ford benchmark



Figure 14: Prim versus Kruskal benchmark

Intersections	Roads	Dijkstra (ms)	Bellman-Ford (ms)	Prim / Kruskal (ms)
10	24	0.2824	0.3383	0.1408 / 0.0944
25	61	0.5349	1.0711	0.2371 / 0.1798
50	124	2.1288	2.8290	0.8748 / 0.3005
100	249	4.5508	8.2699	2.0680 / 0.7490

Algorithm	Complexity	Best used for
BFS / DFS	O(V+E)	Reachability; BFS for fewest junctions, DFS for exploring
Dijkstra	O((V+E) log V)	Fast live navigation with non-negative weights
Bellman-Ford	O(VE)	What-if analysis; handles negative weights
Prim	O(E log V)	Dense networks
Kruskal	O(E log E)	Sparse networks (faster in these tests)

8. Conclusion

This project shows that a city road network can be managed effectively with graph algorithms. Dijkstra found the shortest route (cost 22) and was about twice as fast as Bellman-Ford, which in turn made it possible to test congestion and see the route change to cost 26. BFS and DFS confirmed that all intersections are connected, and Prim and Kruskal produced the same minimum road network of cost 36, with Kruskal faster on these sparse graphs.

Overall, graph algorithms give fast, explainable and optimal decisions for navigation, rerouting and infrastructure planning. The system can be extended with one-way roads, A* search, live traffic data, real map import from OpenStreetMap and time-dependent weights.